CS59001 : Design Lab Report
Room Booking System

Mihir Mallick
Roll Number: 21CS30031

Contents

1 Introduction **4**

2 Problem Statement **4**

3 Objectives **4**

4 Technology Stack **5**
4.1 Backend 5
4.2 Frontend 5
4.3 Storage Model 5

5 High-Level System Overview **5**
5.1 Frontend Responsibilities 6
5.2 Backend Responsibilities 6

6 System Architecture **6**
6.1 Project Structure 6
6.2 Architectural Style 7

7 Core Design Components **7**
7.1 Room Model 7
7.2 User Model 7
7.3 Booking Model 8
7.4 BookingSystem Singleton 8

8 Initialization and Default Setup **9**

9 Implemented Features **9**
9.1 Feature 1: User Registration 9
9.2 Feature 2: User Login 10

9.3 Feature 3: Asymmetric Authentication Rate Limiting 10
9.4 Feature 4: Forgot Password / Password Reset 11
9.5 Feature 5: Search Room Availability 11
9.6 Feature 6: Booking a Room 12
9.7 Feature 7: Overlap Detection 13
9.8 Feature 8: My Bookings View 13
9.9 Feature 9: User-Initiated Booking Cancellation 14
9.10 Feature 10: Admin Dashboard 14

10 Frontend Design and User Experience **15**
10.1 Design Goals 15
10.2 Authentication-State-Aware Interface 15
10.3 Default Time Selection 16
10.4 User Feedback 16

11 Frontend Screenshots and Feature Walkthrough **16**
11.1 Landing Page 16
11.2 User Registration Flow 17
11.3 Room Availability Search 18
11.4 User Booking View 18
11.5 Administrator Dashboard 19

12 Security and Reliability Considerations **19**
12.1 Password Hashing 19
12.2 JWT Authentication 19
12.3 Role-Based Authorization 19
12.4 Authentication Rate Limiting 19
12.5 Email Normalization 19
12.6 Email Enumeration Protection 19
12.7 Concurrency Control 20
12.8 Non-Conflict Between Rate Limiting and Concurrency Control 20

13 Edge Cases and Special Conditions **20**

14 API Summary **21**

15 Software Engineering Perspective **22**
15.1 Modularity 22
15.2 Single Responsibility 22
15.3 Extensibility 22
15.4 Correctness Under Concurrency 22
15.5 User-Centric Design 22

16 Design Lab Perspective **23**
16.1 Need Identification 23
16.2 Functional Decomposition 23
16.3 Interaction Design 23
16.4 Failure-Oriented Thinking 23
16.5 Prototype-to-Product Thinking 23

17 Limitations of the Current Implementation **23**

18 Future Improvements **24**

19 Conclusion **24**

1. Introduction

This report presents the design, implementation, and analysis of a **Room Booking System** developed as part of the Design Lab course. The project is a web-based application intended to simplify the process of checking room availability, making bookings, managing user accounts, and handling booking administration through a role-based interface.

The system was designed from a Software Engineering perspective with emphasis on modularity, usability, authentication, concurrency handling, and separation of concerns. From the perspective of Design Lab, the work demonstrates the process of converting a practical problem into a structured software product with clear user roles, interfaces, workflows, and edge-case handling.

2. Problem Statement

In many institutional and organizational settings, rooms such as discussion rooms, meeting rooms, and conference rooms are shared resources. Manual coordination of these resources often leads to:

- Double bookings,
- Lack of real-time visibility into availability,
- Difficulty in tracing who made which booking,
- Poor user experience while making reservations,
- No clear administrative control over users and bookings.

The objective of this project was to build a lightweight yet functionally complete room booking system that allows:

- users to register and log in,
- users to search available rooms for a time slot,
- authenticated users to make bookings,
- users to see and cancel their own bookings,
- administrators to inspect all users and all bookings and force-cancel bookings when necessary.

3. Objectives

The main objectives of the project were:

- To design a clean and responsive web interface for room search and booking.

- To implement a backend API supporting availability checks, booking, authentication, and admin functions.
- To enforce role-based access control for normal users and administrators.
- To prevent inconsistent booking states such as overlapping bookings.
- To account for practical edge cases such as invalid times, past bookings, duplicate users, unauthorized cancellation, and concurrent booking attempts.
- To keep the implementation modular and easy to extend in future iterations.

4. Technology Stack

4.1. Backend

- Node.js
- Express.js
- TypeScript
- JSON Web Tokens (JWT) for authentication
- bcrypt for password hashing
- express-rate-limit for throttling authentication requests
- async-mutex for concurrency control during booking
- uuid for unique booking and user identifiers

4.2. Frontend

- HTML5
- CSS3
- Vanilla JavaScript

4.3. Storage Model

- In-memory storage for rooms, users, and bookings

5. High-Level System Overview

The Room Booking System follows a client-server architecture.

5.1. Frontend Responsibilities

The frontend is responsible for:

- rendering the interface,
- collecting input from the user,
- invoking backend APIs,
- updating the interface based on authentication state,
- displaying booking results, user bookings, admin dashboards, and notifications.

5.2. Backend Responsibilities

The backend is responsible for:

- exposing REST-style API endpoints,
- validating input data,
- managing users and bookings,
- checking room availability,
- enforcing authentication and authorization,
- handling concurrency during room booking,
- returning structured JSON responses.

6. System Architecture

6.1. Project Structure

The source code is organized into logical modules:

- `src/index.ts`: application entry point
- `src/config.ts`: configuration and JWT settings
- `src/models/`: core domain models such as rooms, users, and booking system
- `src/routes/`: API route handlers
- `src/middleware/`: authentication and admin authorization middleware
- `src/utils/`: utility logic such as room-level lock management
- `frontend/`: HTML, CSS, and JavaScript for the client side

6.2. Architectural Style

The system follows a modular layered design:

- **Presentation Layer**: HTML/CSS/JS frontend
- **Routing Layer**: Express routes handling requests and responses
- **Business Logic Layer**: `BookingSystem` singleton class
- **Utility Layer**: JWT configuration, authentication middleware, mutex-based locking
- **Data Layer**: in-memory arrays for rooms, users, and bookings

This separation helps maintain clarity and supports future migration to persistent databases and richer UI frameworks.

7. Core Design Components

7.1. Room Model

The project uses an abstract `Room` class and concrete derived room types:

- `StandardRoom`
- `ConferenceRoom`

Each room type provides:

- room number,
- capacity,
- room type.

This object-oriented design makes the system extensible. New room categories such as seminar rooms, labs, or premium cabins can be added without changing the higher-level booking flow.

7.2. User Model

Users are represented with:

- unique ID,
- name,
- email,
- hashed password,

- role.

Two roles are currently supported:

- `USER`
- `ADMIN`

7.3. Booking Model

A booking stores:

- booking ID,
- room ID,
- start time,
- end time,
- client name,
- user ID of the booking creator.

The addition of `userId` allows the system to map bookings to the correct user and enforce ownership checks during cancellation.

7.4. BookingSystem Singleton

The `BookingSystem` class acts as the central service layer. It manages:

- rooms,
- users,
- bookings,
- availability checks,
- booking creation,
- booking cancellation,
- registration and password update operations.

The singleton pattern ensures one shared application state across routes in the current execution context.

8. Initialization and Default Setup

At startup, the backend initializes:

- Room 101 as a standard room,
- Room 102 as a standard room,
- Room 201 as a conference room.

It also seeds one administrator account:

- Email: `admin@spacesync.com`
- Password: `admin123`

This seeded admin enables testing and demonstration of privileged features without requiring manual creation of admin credentials.

9. Implemented Features

9.1. Feature 1: User Registration

Purpose: To allow a new user to create an account and begin using the booking system.

Workflow:

1. User opens the registration modal.
2. User enters name, email, and password.
3. Backend validates required fields.
4. Email is normalized by trimming whitespace and converting to lowercase.
5. Password is hashed using bcrypt.
6. Backend checks if a user with the same normalized email already exists.
7. If valid, user is stored and a JWT token is issued immediately.

Design Choices:

- Auto-login after registration improves usability.
- Password hashing ensures passwords are not stored in plain text.
- Email normalization prevents duplicate accounts caused by case or spacing differences.

Edge Cases Handled:

- Missing name, email, or password
- Duplicate email registration
- Mixed-case or whitespace-padded email inputs

9.2. Feature 2: User Login

Purpose: To allow an existing user to securely authenticate.

Workflow:

1. User enters email and password.
2. Backend validates presence of both fields.
3. Email is normalized.
4. User record is fetched.
5. bcrypt compares the input password with the stored hash.
6. On success, a signed JWT token is returned.

Design Choices:

- JWT-based stateless authentication keeps the backend simple.
- The token contains user ID and role, allowing role-aware access control.

Edge Cases Handled:

- Missing credentials
- Invalid email
- Incorrect password
- Expired or malformed token on subsequent requests

9.3. Feature 3: Asymmetric Authentication Rate Limiting

Purpose: To reduce brute-force and abuse attempts on authentication-related endpoints while avoiding unnecessary restriction on legitimate successful users.

Implementation: The authentication routes are protected using `express-rate-limit`. The limiter is configured with:

- a **15-minute reattempt window**,
- a cap of **20 failed requests per IP** within that window,
- **skipSuccessfulRequests = true**.

Why it is asymmetric: The limiter is **asymmetric** because successful authentication requests are excluded from the quota, while failed requests continue to count toward the limit. This means genuine users who successfully log in or register are not penalized in the same way as repeated failed attempts from potentially malicious actors.

Effect of the 15-minute window: If the failed-attempt threshold is exceeded, the client must wait until the 15-minute window resets before making further authentication attempts.

Edge Cases Handled:

- repeated failed login attempts from the same IP,
- repeated failed or abusive registration attempts,
- repeated forgot-password abuse attempts,
- allowing successful requests without consuming the failure quota.

Design Benefit: This approach is stricter against suspicious behaviour while remaining user-friendly for legitimate successful usage.

9.4. Feature 4: Forgot Password / Password Reset

Purpose: To let users recover access by resetting their password.

Current Scope: This is implemented as a mocked direct reset endpoint rather than an email-based workflow.

Workflow:

1. User enters email and new password.
2. Backend checks whether a user exists.
3. If user exists, password is re-hashed and updated.
4. If user does not exist, the backend still returns a generic success-style response.

Security-Oriented Edge Case Handled: If the user does not exist, the system does not reveal that fact explicitly. This avoids **email enumeration**, where an attacker could probe which emails are registered.

Limitation: Since there is no email verification or OTP, this is a demonstration feature rather than a production-grade reset mechanism.

9.5. Feature 5: Search Room Availability

Purpose: To allow users to search which rooms are available between a given start and end time.

Workflow:

1. User selects start and end time in the UI.
2. Frontend performs preliminary validation.
3. Backend receives the query parameters.

4. Query values are parsed into date objects.
5. Backend filters rooms whose time slots do not overlap with existing bookings.
6. Available rooms are returned and rendered as cards.

Design Choices:

- Availability is computed dynamically instead of being stored redundantly.
- The frontend prevents obviously invalid requests before sending them.

Edge Cases Handled:

- Missing `startTime` or `endTime`
- Invalid date format
- Start time greater than or equal to end time
- Search interval entirely in the past
- No available rooms for the selected slot

9.6. Feature 6: Booking a Room

Purpose: To let an authenticated user reserve a selected room for a valid future time slot.

Workflow:

1. User searches for rooms.
2. User selects a room and opens the booking modal.
3. User enters a client name and confirms.
4. Frontend sends an authenticated booking request.
5. Backend validates room, time range, and availability.
6. Booking is saved with a unique ID and linked to the authenticated user.

Important Design Decision: Room-Level Locking A mutex lock is acquired per room before the booking operation proceeds. This means that if multiple requests try to reserve the same room simultaneously, they are serialized. Only one request can complete the booking decision and insertion at a time for that room.

Why This Matters: Without locking, two concurrent requests might both see the room as available and both create bookings, causing a race condition and double booking.

Edge Cases Handled:

- Start time greater than or equal to end time

- Attempt to book in the past
- Non-existent room ID
- Room no longer available for that slot
- Unauthenticated booking attempt
- Near-simultaneous booking attempts for the same room

9.7. Feature 7: Overlap Detection

Purpose: To ensure that a room cannot be booked by two parties for overlapping intervals.

Logic Used: The backend checks whether the requested interval overlaps with any existing booking for the same room.

The overlap cases handled include:

- new booking starts inside an existing booking,
- new booking ends inside an existing booking,
- new booking fully encloses an existing booking.

Practical Effect: This guarantees consistency of room schedules and is one of the core correctness properties of the system.

9.8. Feature 8: My Bookings View

Purpose: To allow users to see the bookings that they have personally created.

Workflow:

1. Authenticated user clicks "My Bookings".
2. Frontend sends an authenticated request.
3. Backend filters all bookings by the user's ID.
4. Only that user's bookings are returned and displayed.

Edge Cases Handled:

- Expired session
- No active bookings for the current user

9.9. Feature 9: User-Initiated Booking Cancellation

Purpose: To let users cancel only their own reservations.

Workflow:

1. User clicks cancel on a booking card.
2. A confirmation modal is shown.
3. Frontend calls the delete endpoint with the booking ID.
4. Backend checks if the booking exists.
5. Backend verifies that the booking belongs to the requesting user.
6. If authorized, the booking is removed.

Design Choices:

- Confirmation modal reduces accidental deletion.
- Ownership verification enforces access control.

Edge Cases Handled:

- Cancellation of non-existent booking
- Attempt to cancel another user's booking
- Expired or invalid session token

9.10. Feature 10: Admin Dashboard

Purpose: To provide administrative visibility into the overall system state.

Admin Capabilities:

- View all users
- View all active bookings
- Force-cancel any booking

Workflow:

1. Admin logs in.
2. Frontend exposes the Admin button only for admin role.
3. Admin dashboard loads all users and all bookings.
4. Admin can force-cancel a booking through the dashboard.

Security Design: All admin routes are protected by:

- token authentication middleware,
- admin-role authorization middleware.

Edge Cases Handled:

- Non-admin attempting to access admin routes
- Admin cancel request for a booking ID that does not exist

10. Frontend Design and User Experience

10.1. Design Goals

The frontend was built with a modern, minimal, and visually polished style. The design uses:

- glassmorphism-inspired cards and modals,
- soft contrast and modern typography,
- modal-driven interactions for focused actions,
- toast notifications for feedback,
- responsive card layouts for room and booking listings.

10.2. Authentication-State-Aware Interface

The frontend dynamically updates itself based on whether the user is:

- not logged in,
- logged in as a normal user,
- logged in as an admin.

Examples:

- login/register buttons are hidden after authentication,
- user greeting appears after login,
- admin dashboard button is visible only to admins,
- results and protected views are hidden after logout.

10.3. Default Time Selection

When the page loads:

- the start time defaults to the current hour,
- the end time defaults to one hour later.

  This improves usability and reduces the amount of manual input required.

10.4. User Feedback

The frontend includes:

- loader spinner while fetching availability,
- success and error toasts,
- inline auth error messages,
- modal confirmation before destructive actions.

11. Frontend Screenshots and Feature Walkthrough

This section presents selected screenshots of the implemented user interface in order to demonstrate the visual design, interaction flow, and role-based features of the Room Booking System.

11.1. Landing Page

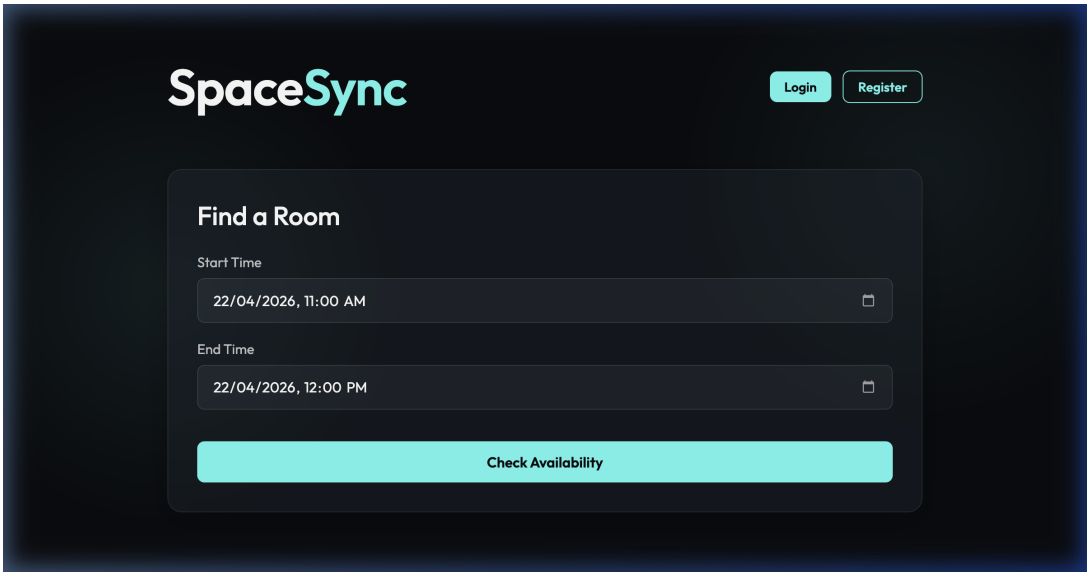

Figure 1: Initial landing page of the Room Booking System showing the SpaceSync interface and the room search form.

11.2. User Registration Flow

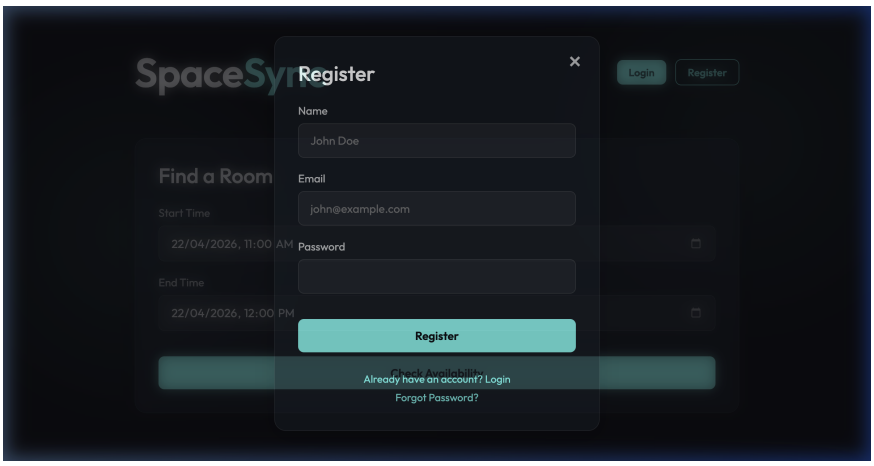

Figure 2: Registration modal for new user onboarding with fields for name, email, and password.

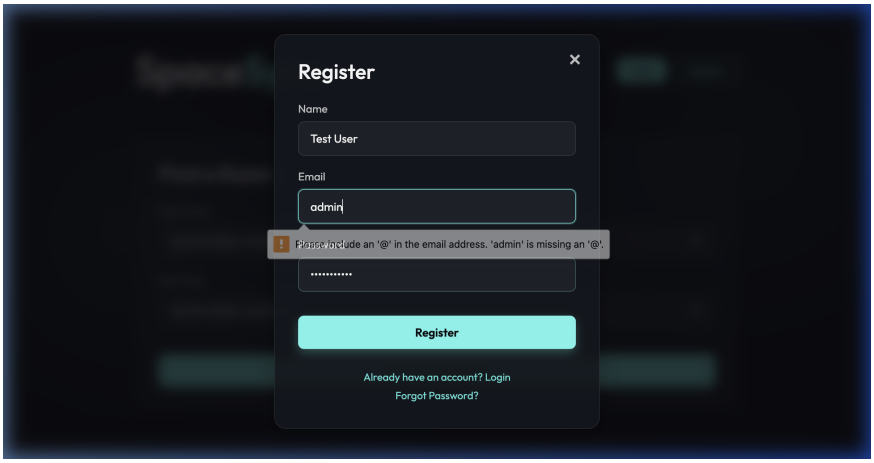

Figure 3: Example of frontend validation feedback for invalid email input during registration.

11.3. Room Availability Search

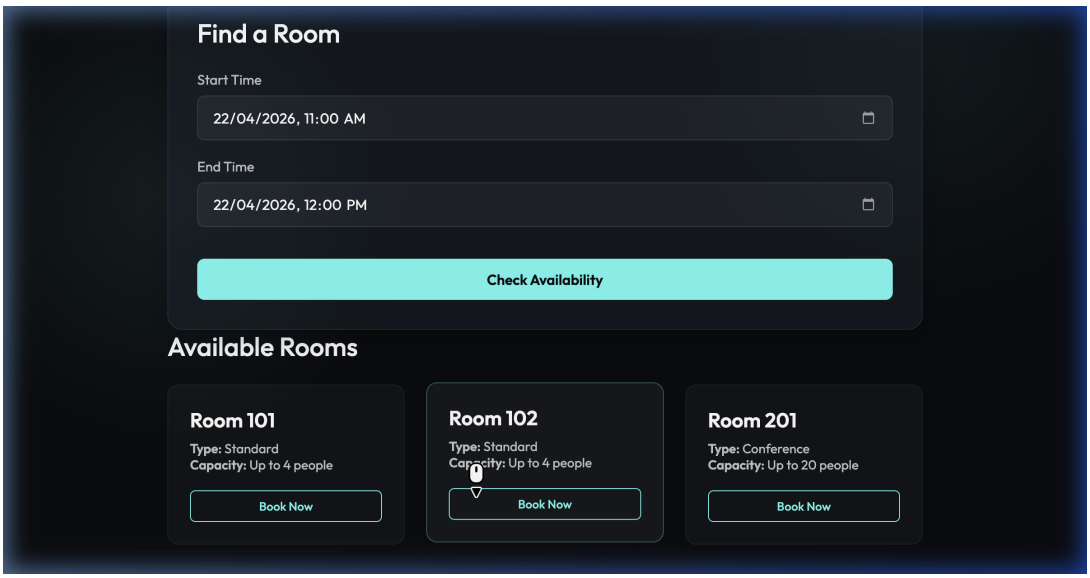

Figure 4: Room availability results after a successful search, showing room type, capacity, and booking action.

11.4. User Booking View

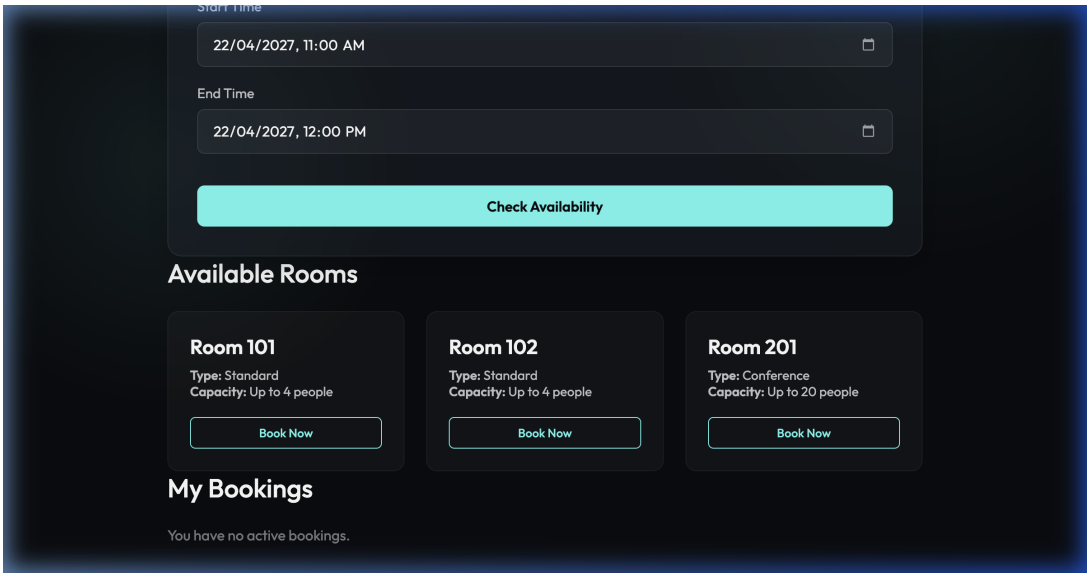

Figure 5: Authenticated user view showing available rooms and the personal bookings section.

11.5. Administrator Dashboard

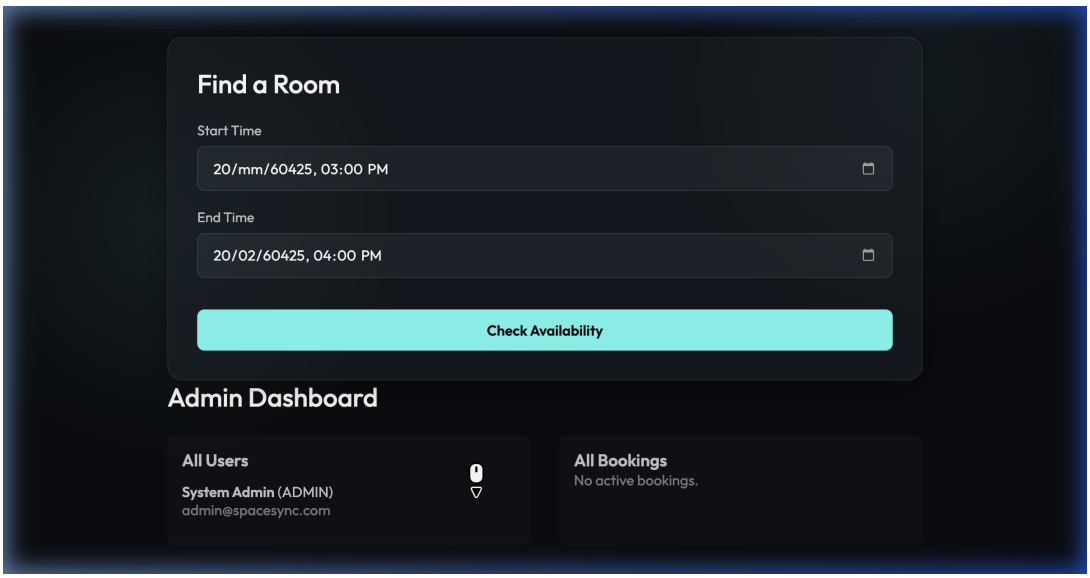

Figure 6: Administrator dashboard showing global visibility over users and bookings.

12. Security and Reliability Considerations

12.1. Password Hashing

Passwords are stored only as bcrypt hashes, reducing damage in case of accidental data exposure.

12.2. JWT Authentication

Protected endpoints require a bearer token. Invalid or expired tokens are rejected.

12.3. Role-Based Authorization

Administrative actions are strictly restricted to users with the admin role.

12.4. Authentication Rate Limiting

The login, registration, and password reset routes are guarded against excessive failed attempts.

12.5. Email Normalization

Lowercasing and trimming emails prevent duplicate account creation for logically identical addresses.

12.6. Email Enumeration Protection

The forgot-password logic avoids revealing whether a particular email exists in the system.

12.7. Concurrency Control

The room-level mutex prevents a race condition during simultaneous booking attempts.

12.8. Non-Conflict Between Rate Limiting and Concurrency Control

The system includes both authentication rate limiting and room-booking concurrency control, and these two mechanisms do not conflict with each other because they operate on different parts of the request lifecycle and solve different classes of problems.

The **rate limiter** is applied only to authentication-related endpoints such as registration, login, and password reset. Its purpose is to reduce abuse, brute-force attempts, and repeated malicious authentication requests from the same IP address.

The **concurrency control** mechanism is applied only to the booking operation. It uses a room-level mutex to serialize concurrent booking attempts for the same room so that race conditions and double bookings cannot occur.

Therefore, the two mechanisms are orthogonal:

- the rate limiter protects **who can keep attempting authentication**,
- the mutex protects **how booking state is modified under concurrent access**.

Since they act at different endpoints and at different logical layers, they do not interfere with one another.

13. Edge Cases and Special Conditions

This project handles multiple practical edge cases, which is an important part of both software design and Design Lab thinking.

Scenario	System Behaviour
Start time is after end time	Request is rejected as invalid.
Booking requested for past time	Request is rejected.
Invalid datetime values	Request is rejected with an error response.
Room ID does not exist	Booking request fails with room-not-found error.
Requested room already booked	Booking request is denied.
Two users try to book same room simultaneously	Mutex lock serializes access and prevents double booking.
Unauthenticated user tries to book or view personal bookings	Request is denied.

User tries to cancel another user's booking	Request is denied.
Non-admin tries to access admin dashboard APIs	Request is denied.
Unknown email in forgot-password flow	Generic success message returned to prevent user enumeration.
User has no bookings	UI shows a dedicated "no bookings" message.
No room available for searched slot	UI shows a dedicated "no rooms available" message.
Expired token on protected request	User is logged out and prompted to log in again.

14. API Summary

Endpoint	**Method**	**Purpose**
`/api/availability`	GET	Returns available rooms for a given time range
`/api/book`	POST	Books a room for an authenticated user
`/api/rooms`	GET	Returns configured rooms
`/api/my-bookings`	GET	Returns bookings belonging to the authenticated user
`/api/bookings/:id`	DELETE	Cancels the authenticated user's booking
`/api/auth/register`	POST	Registers a new user and returns token
`/api/auth/login`	POST	Authenticates user and returns token
`/api/auth/forgot-password`	POST	Mock password reset
`/api/admin/users`	GET	Returns all users for admin
`/api/admin/bookings`	GET	Returns all bookings for admin
`/api/admin/bookings/:id`	DELETE	Force-cancels any booking as admin

15. Software Engineering Perspective

From a software engineering viewpoint, the project demonstrates the following principles:

15.1. Modularity

Business logic, routes, middleware, and utilities are separated into different files and folders.

15.2. Single Responsibility

Each module has a focused purpose:

- middleware handles auth,
- routes handle request/response,
- models capture domain objects,
- the booking system class handles application logic.

15.3. Extensibility

The abstract room model and clear route structure allow future additions such as:

- database integration,
- booking history,
- recurring bookings,
- room amenities,
- approval workflows,
- email notifications,
- calendar integration.

15.4. Correctness Under Concurrency

The use of mutex-based room locking shows attention to multi-user consistency, which is a strong design choice for a booking system.

15.5. User-Centric Design

The interface supports clear flows, fast feedback, and minimal friction for common actions.

16. Design Lab Perspective

From the Design Lab perspective, this project is not merely a coding exercise but a solution design exercise. The following design dimensions are visible:

16.1. Need Identification

The project addresses a real workflow problem: managing shared room resources efficiently.

16.2. Functional Decomposition

The problem was broken into smaller components:

- authentication,
- room discovery,
- reservation,
- booking ownership,
- administration.

16.3. Interaction Design

The modal-based user journey, role-sensitive UI, and feedback mechanisms reflect user-interface planning rather than raw backend implementation alone.

16.4. Failure-Oriented Thinking

The system explicitly handles invalid input, unauthorized actions, time conflicts, and concurrency issues.

16.5. Prototype-to-Product Thinking

Although the current version uses in-memory storage, the structure resembles a scalable product design and can evolve toward a production system.

17. Limitations of the Current Implementation

While functionally strong for a course project, the current system has some limitations:

- Data is stored in memory, so all users and bookings are lost when the server restarts.
- The forgot-password flow is mocked and does not include email verification.
- There is no persistent audit log for booking changes.

- There is no support for room attributes such as equipment, location, or floor.
- There is no advanced booking policy such as maximum duration, buffer intervals, or recurring bookings.
- There is no database-backed transaction support beyond in-memory mutex locking.
- The default admin credentials are suitable only for demonstration and should not be used in production.

18. Future Improvements

The system can be improved in several directions:

- Integrate a database such as PostgreSQL or MongoDB for persistent storage.
- Add email-based password reset using secure tokens.
- Add booking history and activity logs.
- Introduce room filters based on capacity, type, and amenities.
- Add date-range restrictions and institutional policies.
- Add dashboard analytics for room utilization.
- Add calendar views and reminders.
- Improve frontend accessibility and mobile responsiveness further.
- Add automated tests for booking overlap logic, auth flow, and admin actions.

19. Conclusion

The Room Booking System developed in this project successfully demonstrates the design and implementation of a multi-user booking platform with authentication, authorization, room availability computation, safe booking creation, user-specific booking management, and administrative oversight.

The project is especially notable for handling practical correctness issues such as:

- time validation,
- overlapping bookings,
- role-based access,
- booking ownership,
- concurrent booking conflict prevention.

From a software engineering standpoint, the project uses modular organization, secure password storage, token-based authentication, and concurrency control. From a Design Lab standpoint, it reflects structured problem solving, interface design, and failure-aware system thinking.

Overall, this project serves as a meaningful and well-scoped design exercise that can be extended into a more production-ready room scheduling platform with persistent storage and richer policy controls.